

유니티 개인 프로젝트 - 적을 아군으로 쓰는 게임



기술 경험

- 인벤토리
- 거리 기반 적 타겟팅 AI
- AI 아군 유닛 시스템
- 의존성 주입(DI)
- DOTween

YOU - 적을 아군으로 쓰는 게임

‘YOU’ 는 적 몬스터를 기절시키고 아군으로 전환시켜 플레이어를 따라다니는 ‘팔로워’ 유닛으로 활용하는 2D 캐주얼 액션 게임입니다.

사용 기술 : Unity2D, JSON

장르 : 캐주얼 액션

플랫폼 : Windows

엔진 버전 : Unity 2022.3.28f1

GitHub : [a-i-am/Unity2D-CasualAction-DemoProject-YOU-](https://github.com/a-i-am/Unity2D-CasualAction-DemoProject-YOU-)

영상 : <https://youtu.be/DXausfETae0>

핵심 구현 기능

- AI 아군 유닛 시스템
- 거리 기반 적 타겟팅 AI
- AI 아군 공중 대시 공격 & 제자리로 복귀

“적을 기절시켜 아군으로 활용하는 2D 플랫폼어 액션 RPG”

1



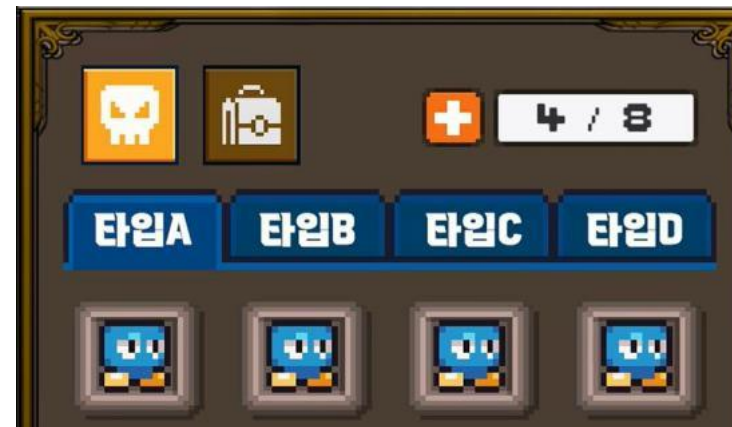
적을 공격해 기절(체력 0) 상태로 만듭니다.

2



쓰러진 적들은 주워서 데려갈 수 있습니다

3



주운 적들은 인벤토리에서 확인할 수 있습니다.

4

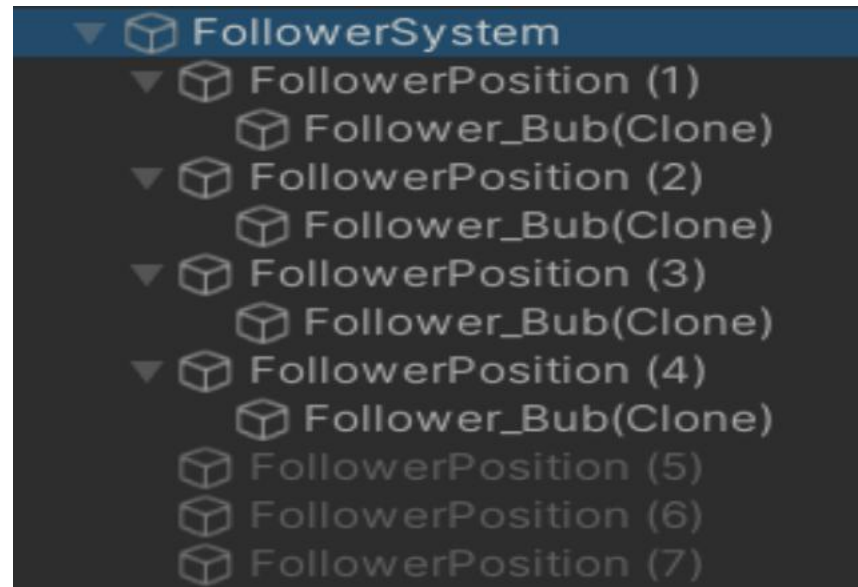


수집한 적을 소환하면 플레이어를 뒤따르는 아군 유닛이 됩니다.

* PC 환경에서 URL을 클릭하시면 해당 내용에 대한 Github 코드 페이지로 이동됩니다.

팔로워 순차 소환 및 등록 : 큐(Queue) 를 통한 개체 순차 등록

Queue의 FIFO(First In First Out) 특성을 통해 팔로워 개체가 지정된 스폰 위치들에 등록된 순서대로 생성될 수 있도록 하였습니다.



```
public void SpawnFollower(Character.CharacterData characterData)
{
    if (emptySpawnQueue.Count == 0 || characterData == null || characterData.characterPrefab == null) return;

    spawnPos = emptySpawnQueue.Dequeue();
    spawnPos.gameObject.SetActive(true);

    follower = Instantiate(characterData.characterPrefab, spawnPos.position, Quaternion.identity, spawnPos);
    Debug.Log("팔로워 생성");
}
```

(Github) FollowerSpawner.cs (33번줄~48번줄)

<https://github.com/a-i-am/Unity2D-CasualAction-DemoProject-YOU-/blob/main/Assets/Scripts/Objects/Characters/Followers/Objects/FollowerSpawner.cs>

* PC 환경에서 URL을 클릭하시면 해당 내용에 대한 Github 코드 페이지로 이동됩니다.

List & Set 및 HashSet 활용 거리 기반 적 타겟팅 AI

아군 유닛이 감지한 적을 공격할 때, 아군끼리 동시에 같은 적을 고르지 않되, 각 개체와 가장 가까운 거리의 적을 우선적으로 공격할 수 있게 하였습니다.

기능 요구사항

아군 유닛	(1) 순서대로 타겟을 할당하거나 행동을 정할 수 있다. (2) 게임 도중에 추가/삭제될 때 적을 타겟팅 할 유닛 개체 목록에서 빠져야한다.	→	List<T>	순서 보장, 중간 삽입/삭제
적 개체들	(1) 탐지 범위에 들어온 적만 타겟팅 후보에 등록할 수 있다. (2) 타겟팅 후보에 등록된 적이 중복 등록되지 않게 해야한다. (3) 탐지범위에서 벗어난 적은 타겟 후보에서 빠르게 제외해야 한다.	→	HashSet<T>	삽입 값 자동 정렬
타겟팅 후보	(1) 가장 가까운 적이 우선순위가 되게 정렬되어야 한다. (2) 같은 거리에 있는 적인 경우, 또다른 타겟팅 우선순위 기준이 있어야 한다.	→	SortedSet<T1, T2>	중복 방지 + 빠른 탐색

- 기절한 적은 타겟팅 후보에 포함되지 않도록 하는 등, 게임에서 실시간으로 다수의 유닛을 다룰 때 탐색 비용이 낮도록 설계했습니다.

* PC 환경에서 URL을 클릭하시면 해당 내용에 대한 Github 코드 페이지로 이동됩니다.

Dictionary<T> + TryGetComponent() 로 GetComponent 데이터 캐싱

몬스터 개체와 팔로워 시스템 간 자원(위치값, 이벤트, 타겟팅 데이터 등)이 많아 공유할 멤버만 인터페이스로 명확하게 구분 지었습니다.

```
public class FollowerController : MonoBehaviour, IFollowerTargetReceivable, IFollowerAttackable
public class TargetingAI : Singleton<TargetingAI>, IFollowerNumberCheck, IEnemyNumberCheck
```

```
public class Enemy : MonoBehaviour
{
    // 인터페이스 참조
    private IEnemyNumberCheck enemyNumberChecker;
    [SerializeField] private EnemyController enemyController;
```

```
public class Follower : MonoBehaviour
{
    [Header("외부 참조")]

    // 인터페이스 참조
    private IFollowerNumberCheck followerNumChecker;
    private IFollowerTargetReceivable followerTargetReceivable;
    private IFollowerAttackable followerAttackable;
```

```
private void Awake()
{
    enemyNumberChecker = TargetingAI.Instance;

    if (enemyNumberChecker != null)
    {
        enemyController.FaintedEvent -= Remove;
        enemyController.FaintedEvent += Remove;
    }
}
```

- 싱글톤과 GetComponent<> 등이 어려웠던 직접적인 클래스 참조방식에서 의존성 주입과 업 캐스팅 방식으로 수정하였습니다.
- 그 덕분에 기존보다 구체 타입 의존을 줄이고 추상 타입에 의존하여 결합도를 낮추어 확장·유지보수를 쉽게 만들 수 있었습니다.

* PC 환경에서 URL을 클릭하시면 해당 내용에 대한 Github 코드 페이지로 이동됩니다.

의존성 주입(DI) + 업캐스팅을 통한 참조 분리

(Github) FollowerSpawner.cs (84번줄~97번줄)

<https://github.com/a-i-am/Unity2D-CasualAction-DemoProject-YOU-/blob/main/Assets/Scripts/Objects/Characters/Followers/Calculator/TargetingAI.cs>

Dictionary는 데이터를 저장 및 조회 시 특정 키에 해당하는 값을 O(1)에 가까운 시간 복잡도로 메모리 위치에 직접 접근할 수 있어, 충돌 발생 때마다 특정 컴포넌트를 검색하는 로직에 적용하였습니다. 또한 캐싱 기법을 활용하여 적 중복 타겟팅을 방지하였습니다.

```
private Dictionary<Collider2D, Enemy> _enemyCache = new Dictionary<Collider2D, Enemy>();
```

```
private Enemy GetEnemyFromCollider(Collider2D collider)
{
    if(collider == null) return null;

    if(_enemyCache.TryGetValue(collider, out Enemy enemy))
    {
        return enemy;
    }

    if (collider.TryGetComponent(out enemy))
    {
        _enemyCache[collider] = enemy;
        return enemy;
    }

    return null;
}
```

- GetComponent와 같은 비용이 적지 않은 작업을 매 프레임 호출 할 때의 성능 저하에 대비해 Dictionary를 활용한 컴포넌트를 캐싱하는 코드를 적용하였습니다.
- 또한 컴포넌트가 존재하지 않을 때의 내부적인 예외처리 비용 없이 컴포넌트의 존재 여부를 확인하기 위해 TryGetComponent()를 사용하였습니다.
- 그 결과 첫 호출 시의 검색 비용만 지불하고 이후 호출은 캐싱으로 대체할 수 있게 되었습니다.

* PC 환경에서 URL을 클릭하시면 해당 내용에 대한 Github 코드 페이지로 이동됩니다.

DOTween 활용 대시 공격 & 리턴 AI

(Github) FollowerSpawner.cs (104번줄~144번줄)

<https://github.com/a-i-am/Unity2D-CasualAction-DemoProject-YOU-/blob/main/Assets/Scripts/Objects/Characters/Followers/Controller/FollowerController.cs>

팔로워 캐릭터의 대시 공격 & 제자리 복귀 시퀀스를 구현하던 중, 대시 후 복귀 지점이 대시 공격 지점과 동 떨어지게 변동되는 문제가 있었습니다.



1. 타겟 결정
2. 대시공격 시작
3. 자신의 고정 위치로 복귀



```
public void DashAndReturn()
{
    if (isDashing && _currentTarget == null && _currentTarget.isFainted) return;
    디버깅(null 체크)
    isDashing = true; // 동작 종료 전 재실행 불가
    // 타겟팅 정보 설정
    _targetPos = TargetPosition;

    // 팔로워 위치 & 움직임 조정
    startY = followerGroupMoving.startY;
    followerGroupMoving.isSineActive = false;

    Sequence seq = DOTween.Sequence();
    seq.Append(transform.DOMove(_targetPos, dashDuration))
        .AppendInterval(0.5f)
        .Append(transform.DOMove(followPos, dashDuration))
        .OnComplete(() =>
        {
            _currentTarget = null;
            followerGroupMoving.isSineActive = true;
            isDashing = false;
            TargetingAI.Instance.ClearTargetHashSet();
        })
        .Play();

    ++dashCount;
}
```

- Transform을 수동 조정하는 방식으로 캐릭터 모션을 구현하면서, 캐릭터의 대시 공격 모션을 부드럽게 연출하기 어렵다는 한계를 느꼈습니다.
- 특히 여러 물리적인 동작들이 동시에 적용되는 경우 동작의 타이밍이나 오브젝트 떨림 등의 버그들이 자주 발생하였습니다.
- 그래서 시작 시점에 목표 좌표를 고정하여 이동 시퀀스를 반복 재생할 수 있는 DOTween을 활용하였습니다.
- 그 결과 중간에 대상 객체의 위치가 변해도 원래 의도한 자리로 정확히 복귀할 수 있게 되었습니다.

* PC 환경에서 URL을 클릭하시면 해당 내용에 대한 Github 코드 페이지로 이동됩니다.

인벤토리 아이템 슬롯 버그 해결

(Github) 리팩토링된 인벤토리 코드 목록

<https://github.com/a-i-am/Unity2D-CasualAction-DemoProject-YOU-/tree/main/Assets/Scripts/Refactoring/Inventory>

인벤토리에 들어간 같은 종류의 아이템 클릭(사용) 시, 어떤 아이템을 클릭하든 가장 늦게 들어온 아이템부터 삭제되는 버그를 해결하였습니다.

아이템 데이터에 대한 1회성 참조 변수에 모든 아이템 데이터를 저장했기 때문에, 어떤 슬롯을 선택하든 같은 주소의 객체가 되는 것이 버그 원인이었습니다.

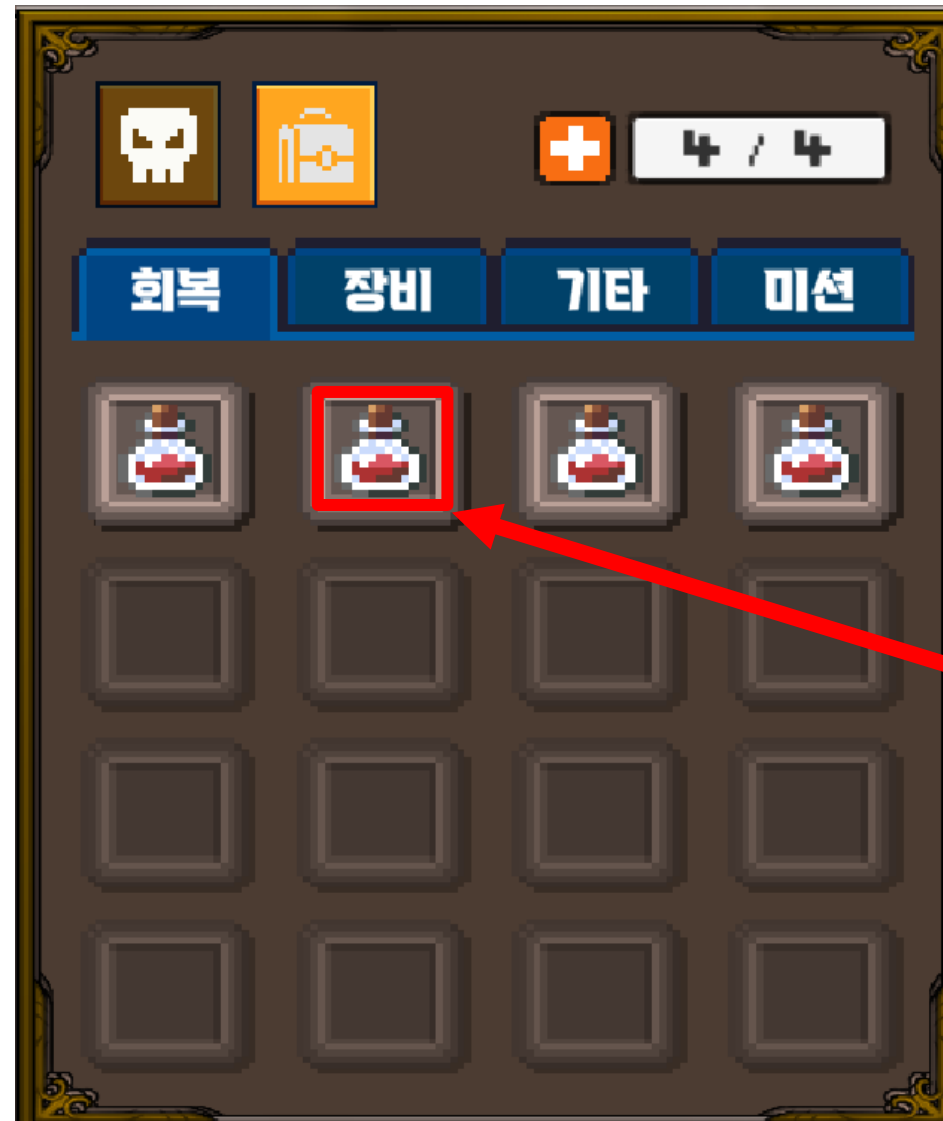
```
[Serializable]
public class ItemMasterData
{
    public int itemID;
    public string type;
    public string itemName;
    public string explain;
    public int maxStack;

    [System.NonSerialized]
    public Sprite itemImage;

    public List<EffectData> effects = new List<EffectData>();
}

[Serializable]
public class ItemInstance
{
    public ItemMasterData masterData { get; private set; }
    public int count;
    public int durability;

    public ItemInstance(ItemMasterData master, int amount)
    {
        this.masterData = master;
        this.count = amount;
        this.durability = 100;
    }
}
```



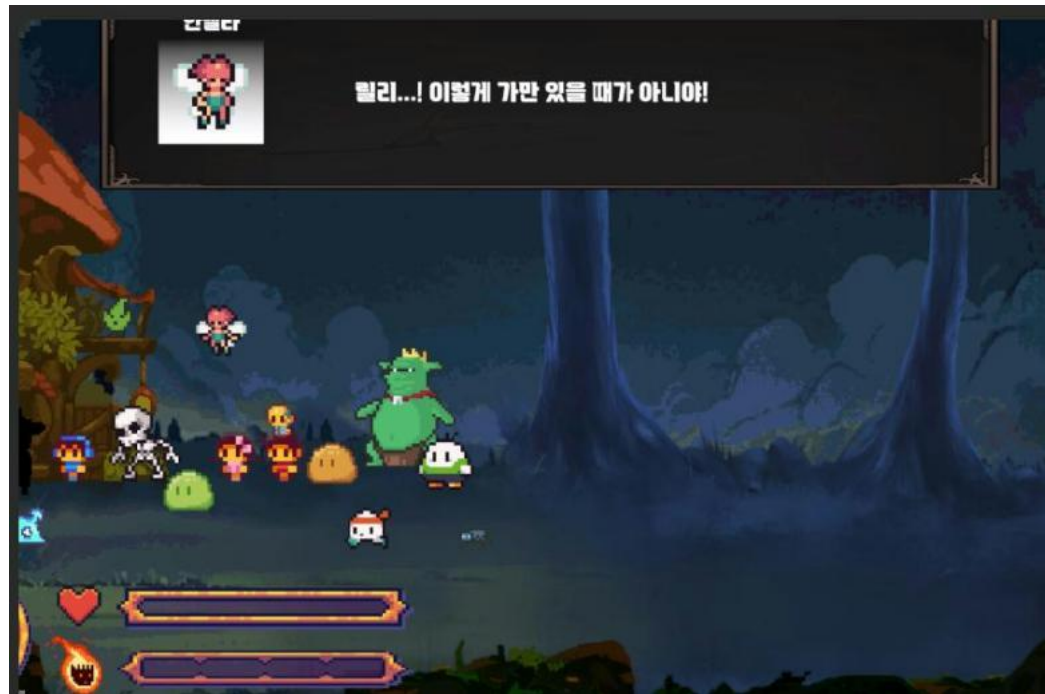
클릭한 슬롯

(Bug) 실제로 아이템이 삭제된 슬롯

- 아이템에 대한 1회성 참조 정보(불변 데이터)와 new 객체로 생성할 정보(가변 데이터)를 구분해 아이템 인스턴스를 생성하였습니다.
- 아이템 획득 시 매번 new ItemInstance를 생성하여 고유한 메모리 참조를 갖게 함으로써, 특정 슬롯만 독립적으로 삭제되도록 수정했습니다.

* PC 환경에서 URL을 클릭하시면 해당 내용에 대한 Github 코드 페이지로 이동됩니다.

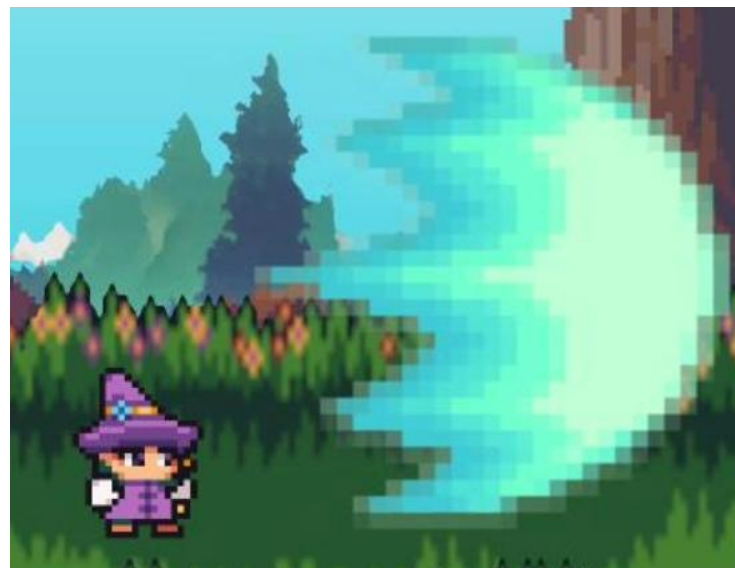
그 외 개발 내역



유니티 Cinemachine 활용 컷 씬



타이밍 별 공격 패턴을 가진 보스 몬스터



플레이어 단계별 차징 필살기